

Project Phase 4 Report

Web Integrations of SQL and NoSQL Databases

Group Number: 22

Group Members:

- Alperen Sarişen 33886
- Hamza Eren İnan 34502

1. Trigger

The trigger implemented in this project is an **AFTER INSERT / AFTER DELETE** trigger on the **Member** table. It automatically logs a message to the **audit_log** table whenever a member is added or removed from the system.

1.1 SQL Script

```
-- Trigger: After INSERT on Member
CREATE TRIGGER after_member_insert
AFTER INSERT ON Member
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (message, created_at)
    VALUES (CONCAT('New member added: ', NEW.mname), NOW());
END;

-- Trigger: After DELETE on Member
CREATE TRIGGER after_member_delete
AFTER DELETE ON Member
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (message, created_at)
    VALUES (CONCAT('Member deleted: ', OLD.mname), NOW());
END;
```

1.2 Web Page Cases and Buttons

The trigger page (**trigger_1.php**) demonstrates the trigger behaviour through interactive forms. Two cases are covered:

Case 1 – Add Member: The user fills in Member ID, Name, Age, and Address, then clicks *Add Member*. This fires the AFTER INSERT trigger, which writes a log entry confirming the new member.

Case 2 – Delete Member: The user selects an existing member from a dropdown and clicks *Delete Member*. This fires the AFTER DELETE trigger, which writes a log entry confirming the deletion.

After each action the **Audit Log** table at the bottom of the page updates automatically to show the trigger output.

2. Stored Procedure

The stored procedure implemented is **EnrollMember**. It enrolls a gym member into a class and returns a result message indicating success or failure (e.g. if the member is already enrolled).

2.1 SQL Script

```
DELIMITER $$

CREATE PROCEDURE EnrollMember(IN p_mid INT, IN p_cid INT)
BEGIN
    DECLARE already_enrolled INT DEFAULT 0;

    SELECT COUNT(*) INTO already_enrolled
    FROM Enrolled
    WHERE mid = p_mid AND cid = p_cid;

    IF already_enrolled > 0 THEN
        SELECT 'Member is already enrolled in this class.' AS ProcedureResult;
    ELSE
        INSERT INTO Enrolled (mid, cid, enrollment_date)
        VALUES (p_mid, p_cid, CURDATE());
        SELECT 'Member successfully enrolled.' AS ProcedureResult;
    END IF;
END$$

DELIMITER ;
```

2.2 Input Parameters and Web Page

The procedure page (**procedure_1.php**) provides two dropdown menus:

Parameter 1 – Member (mid): A dropdown listing all members from the Member table.

Parameter 2 – Class (cid): A dropdown listing all classes from the Class table.

After clicking *Execute EnrollMember Procedure*, the page displays the procedure's return message and refreshes the current enrollments table at the bottom.

3. MongoDB Support Ticket System

All support ticket data is stored in a MongoDB collection (**gym_db.tickets**). Each document contains: **username**, **message**, **created_at** (string), **status** (boolean: true = active, false = resolved), and **comments** (array).

3.1 Query 1 – Fetch All Active Tickets (Admin)

Used in **admin/index.php**. Retrieves every ticket with status = true so the admin can see all open requests.

```
$filter = ['status' => true];
$query = new MongoDB\Driver\Query($filter, []);
$cursor = $manager->executeQuery('gym_db.tickets', $query);
```

3.2 Query 2 – Fetch Active Tickets by Username (User)

Used in **user/tickets.php**. Filters active tickets belonging to the selected user so each user sees only their own tickets.

```
$filter = ['status' => true, 'username' => $selectedUser];
$query = new MongoDB\Driver\Query($filter, []);
$cursor = $manager->executeQuery('gym_db.tickets', $query);
```

3.3 Query 3 – Fetch Single Ticket by ID

Used in both **ticket_detail.php** files (user and admin). Retrieves one ticket by its MongoDB ObjectId to display its full details and comments.

```
$filter = ['_id' => new MongoDB\BSON\ObjectId($ticketId)];
$query = new MongoDB\Driver\Query($filter);
$cursor = $manager->executeQuery('gym_db.tickets', $query);
$ticket = current($cursor->toArray());
```

3.4 Query 4 – Create New Ticket

Used in **user/create_ticket.php**. Inserts a new ticket document with the user's username, message, current timestamp, status true, and an empty comments array.

```
$bulk = new MongoDB\Driver\BulkWrite;
$document = [
    'username' => $_POST['username'],
    'message' => $_POST['message'],
    'created_at' => date('Y-m-d H:i:s'),
    'status' => true,
    'comments' => []
];
$bulk->insert($document);
$manager->executeBulkWrite('gym_db.tickets', $bulk);
```

3.5 Query 5 – Add Admin Comment

Used in **admin/ticket_detail.php**. Pushes a new comment object (username = 'admin', comment text, timestamp) into the ticket's comments array.

```
$bulk->update(
    ['_id' => new MongoDB\BSON\ObjectId($ticketId)],
    ['$push' => ['comments' => [
        'username' => 'admin',
        'comment' => $commentText,
        'created_at' => date('Y-m-d H:i:s')
    ]]]
);
$manager->executeBulkWrite('gym_db.tickets', $bulk);
```

3.6 Query 6 – Resolve (Close) Ticket

Used in **admin/ticket_detail.php**. Sets the ticket's status field to false, which removes it from both the user and admin active ticket views.

```
$bulk->update(
    ['_id' => new MongoDB\BSON\ObjectId($ticketId)],
    ['$set' => ['status' => false]]
);
$manager->executeBulkWrite('gym_db.tickets', $bulk);
```

4. Support Ticket System Workflow

User flow: A user visits *tickets.php*, enters their username to see active tickets, can click *View Details* to read comments, and can create new tickets via *create_ticket.php*.

Admin flow: The admin visits *admin/index.php* to see all active tickets from all users. From the detail view the admin can add a comment (posted as 'admin') or click *Deactivate/Resolve Ticket* to close it. Resolved tickets disappear from both views.

Design decisions: No authentication layer was added as the project spec indicates a lightweight prototype. The username field on the user side acts as a simple identifier. Passwords and session management are out of scope for this phase.